

KV Cache Service 技术报告：

从推理中间态到企业知识资产

Nanamiqa

2026 年 8 月 10 日

摘要

随着大语言模型上下文窗口持续扩展，长文档问答的主要成本逐渐从“知识能否放入上下文”转向“相同上下文是否必须被反复计算”。本文给出开源系统 KV CACHE SERVICE 0.3.0 的技术报告，完整说明其问题定义、总体架构、核心实现、工程效果与应用方法。系统面向公共长前缀的重复计算问题，将注意力 Key/Value (KV) 状态封装为可持久化、可治理、可跨请求复用的服务对象。系统提供两条实现路径：Transformers 后端以 safetensors 保存原始 KV 张量，用于单机语义验证；vLLM 后端通过缓存亲和路由复用前缀，并借助 LMCache 将缓存扩展到 CPU、NVMe 与远端层级，用于多 GPU 服务。本文同时给出冷路径与热路径的时延、存储及复用盈亏模型，说明系统的直接效果是减少重复 prefill、降低首 token 延迟并把可预测计算前移到离线时段，而不是减少解码阶段的逐 token 计算。

在此基础上，本文提出四项产业判断：第一，KV Cache 可作为与模型版本绑定的“派生知识资产”，对稳定知识进行预解析和存储；第二，在知识集合可完整容纳于模型上下文且更新频率较低时，长上下文 KV Cache 可减少重复 token 传输与预填充成本，并在部分任务中替代检索步骤、保留全局上下文，但不能突破模型上下文窗口，也不应被笼统视为 RAG 的替代品；第三，企业可在闲时构建和下沉缓存，在忙时将 GPU 主要用于新增后缀与生成；第四，应用需要转向“缓存原生”的场景工程，即主动稳定前缀、规范模板、管理版本与权限，并围绕命中率设计知识包。本文最后给出企业落地架构、实验评估方案及安全、兼容性、时效性等风险边界。

关键词：大语言模型；KV Cache；长上下文；企业知识资产；推理服务；RAG；算力调度

目录

1 引言	3
2 技术背景与问题定义	3
2.1 KV Cache 与重复 prefill	3
2.2 KV Cache 的容量成本	4
2.3 与 RAG 的问题边界	4

目录	2
3 系统架构与实现	4
3.1 总体架构	4
3.2 Transformers 原始 KV 持久化路径	4
3.3 vLLM 与 LMCache 生产路径	5
3.4 工程能力与测试覆盖	5
4 系统效果与经济性分析	6
4.1 直接效果：降低重复计算与 TTFT	6
4.2 效果不覆盖的部分	6
5 从 KV Cache 到企业知识资产	6
5.1 资产定义：从原始文档到模型绑定的派生制品	6
5.2 价值：把重复理解转化为一次预解析	7
6 长上下文问答：替代 RAG 的条件与边界	7
6.1 “绕过 token 限制”的严格解释	7
6.2 无需 RAG 的适用条件	7
6.3 KV Cache、RAG 与混合方案比较	8
7 闲时算力向忙时能力的转换	8
7.1 时间迁移机制	8
7.2 调度策略	9
8 缓存原生的场景工程	9
9 企业落地参考架构	9
10 评估方案	10
11 风险、限制与治理要求	10
12 结论	11

1 引言

大语言模型推理通常分为两个阶段：首先，模型对输入提示词执行并行的预填充（prefill），为每层注意力生成 Key/Value 状态；随后，模型在解码（decode）阶段逐 token 生成答案，并复用此前状态。若大量请求共享相同的制度文档、代码仓库、合同、系统提示词或工具定义，常规服务仍会为每个请求重复计算相同前缀。当前缀达到数万乃至数十万 token 时，这一重复计算会显著抬高首 token 延迟（Time to First Token, TTFT）与 GPU 成本。

跨请求 KV Cache 复用的核心思想，是把模型完成前缀 prefill 后得到的内部状态保存为缓存制品，后续请求只计算新增问题或动态后缀。相关研究已表明，预计算注意力状态可以明显降低长提示词的 TTFT，但收益取决于模型、硬件、存储层级、前缀长度与复用率 [4, 5]。因此，真正的工程问题不只是“能否缓存”，还包括缓存身份、模型兼容性、持久化一致性、多租户隔离、容量治理、亲和路由和失效策略。

为解决上述问题，KV CACHE SERVICE 实现了面向本地与私有化大语言模型推理的持久化 KV Cache 控制面与推理网关 [1]。本文围绕系统设计与应用总结三项主要工作：

1. 设计并实现由 Transformers 原始张量路径与 vLLM/LMCache 生产路径组成的双后端架构，以及缓存生命周期、租户隔离、容量治理和亲和路由能力；
2. 给出缓存复用的时延与经济性模型，明确系统已经实现的功能、可验证的工程效果及仍需在目标环境中测试的性能指标；
3. 从知识资产、长上下文问答、闲忙算力转换和缓存原生场景工程四个方面，总结系统的企业部署方法与适用边界。

2 技术背景与问题定义

2.1 KV Cache 与重复 prefill

设一层自注意力接收隐藏状态 X ，其 Key 与 Value 为

$$K = XW_K, \quad V = XW_V. \quad (1)$$

在因果语言模型中，位置 i 只能关注不晚于 i 的 token。因此，对于完全相同且位置一致的前缀，已经计算出的 K, V 可以被后续解码或后续请求精确复用，而无需再次对该前缀执行 prefill。PagedAttention 等工作主要解决请求内 KV 内存的高效管理与共享 [3]；Prompt Cache、LMCache 等工作则进一步讨论跨请求、跨设备或跨引擎的复用 [4, 5]。

令公共前缀长度为 n ，动态后缀长度为 m ，生成长度为 g 。忽略网络抖动后，普通请求的时延可近似写为

$$T_{\text{cold}} = T_{\text{tokenize}}(n + m) + T_{\text{prefill}}(n + m) + T_{\text{decode}}(g). \quad (2)$$

缓存命中后的时延为

$$T_{\text{hot}} = T_{\text{lookup}} + T_{\text{load}}(B_n) + T_{\text{tokenize}}(m) + T_{\text{prefill}}(m) + T_{\text{decode}}(g), \quad (3)$$

其中 B_n 是前缀 KV Cache 的字节数。由此可见，主要收益来自用查找与数据搬运替代长度为 n 的重复 prefill；解码每个输出 token 的成本通常不会被消除。若缓存位于 GPU HBM, T_{load} 很小但容量昂贵；若位于 CPU、NVMe 或远端存储，容量更大，但收益取决于 I/O 与 CPU-GPU 带宽。

2.2 KV Cache 的容量成本

对于层数 L 、KV 头数 H_{kv} 、每头维度 d_h 、token 数 n 、每元素字节数 s 和批量 b ，未压缩 KV Cache 的理论大小约为

$$B_{KV} = 2LH_{kv}d_hnbs, \quad (4)$$

其中系数 2 分别对应 Key 与 Value。该式揭示了本技术的本质：它不是无成本地“记住知识”，而是把重复计算转换为与 token 数线性增长的存储及搬运成本。KV CACHE SERVICE 文档给出的一个 GQA 示例约为 56 KiB/token，128K token 约需 7 GiB，且尚未包括元数据和运行时开销 [1]。

2.3 与 RAG 的问题边界

RAG 将外部语料建立为可检索的非参数化记忆，并针对问题选择少量相关片段注入上下文 [6]。KV Cache 则保存某个精确前缀在特定模型中的内部状态，本身不负责检索、来源引用或知识更新。两者解决的问题不同：RAG 解决“大语料中选什么”，KV Cache 解决“已选且反复出现的上下文是否需要重算”。当完整知识可放入上下文时，KV Cache 能避免检索漏召回并保留跨段落关系；但长上下文模型仍可能出现“中间信息利用不足”等现象 [7, 8]。因此，“更长窗口”不自动等于“更可靠的全局理解”。

3 系统架构与实现

3.1 总体架构

本文报告的 KV CACHE SERVICE 开源版本为 0.3.0。系统通过 FastAPI 提供缓存创建、查询、删除、清理和 OpenAI 风格 Completion/SSE 接口；HTTP 层只依赖统一的 KVCacheBackend 抽象，因此可在不改变上层 API 的情况下切换本地 Transformers、vLLM 或自定义后端 [1]。其逻辑结构如图1所示。

3.2 Transformers 原始 KV 持久化路径

Transformers 后端面向单机验证、研究与自定义后端开发。其主要流程为：对文本进行精确分词；检查模型上下文上限；按指定 token 数分块 prefill；累计每层 past_key_values；基于租户、模型指纹和 token 序列生成内容寻址的 cache_id；最后把每层 K/V、前缀 token IDs 及前缀末端的下一 token logits 写入 safetensors。命中时，系统验证模型指纹、过期时间、文件校验和和前缀一致性，再把张量恢复到推理设备，只处理新增后缀。

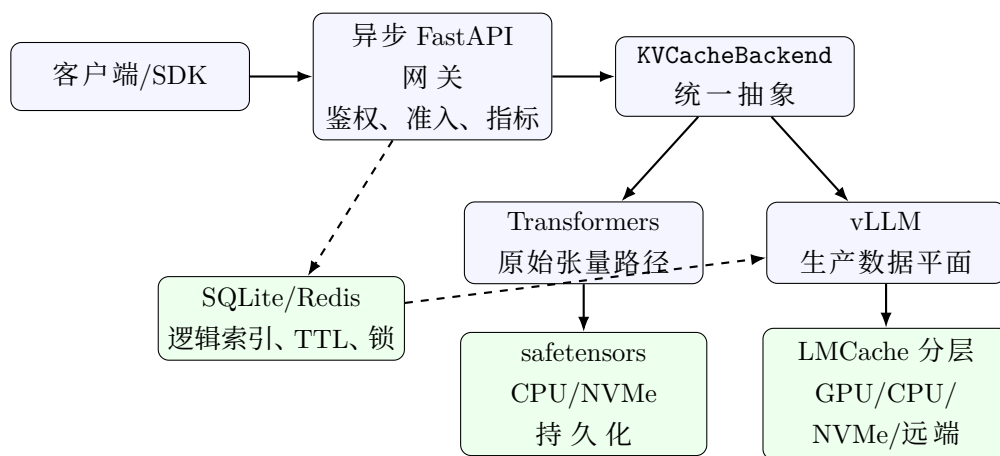


图 1: KV CACHE SERVICE 的双后端与控制面架构

该路径体现了“知识状态制品化”的最直接形式：缓存目录包含 `metadata.json` 与 `tensors.safetensors`。存储层先写临时目录，再通过原子替换发布；默认校验 SHA-256，并使用进程锁和文件锁协调读写删除。TTL、容量上限、LRU 和主动 prune 组成基本生命周期治理。safetensors 避免了任意 Python 对象反序列化，但并不意味着张量不含敏感信息。

3.3 vLLM 与 LMCache 生产路径

生产路径不让网关直接搬运大型 KV 文件，而是将物理 KV block 交给 vLLM 和 LMCache 管理。网关保存租户隔离的逻辑前缀记录，并以 `cache_id` 为亲和键，将预热和后续请求尽量发送到相同 vLLM 副本。路由选择综合 rendezvous affinity、在途负载、EWMA 延迟和逐副本熔断；Redis 可在多个网关之间共享索引、TTL 和分布式预热锁。LMCache 则负责把缓存扩展到 CPU、NVMe 或远端层级。这一分工与相关研究提出的“从 GPU 中抽取 KV，并在不同查询和引擎间共享”的方向一致 [5]。

值得注意的是，vLLM 路径的“创建缓存”实质上是向上游发送一次确定性预热请求，并保存逻辑身份与前缀；物理命中仍依赖 vLLM 的 Automatic Prefix Caching 与 LMCache。因而，控制面的 `cache_id` 不应被误解为任意模型、任意副本间都可直接读取的通用张量地址。

3.4 工程能力与测试覆盖

除缓存本体外，系统实现了 Bearer Token/API Key、多租户身份参与缓存 ID、全局与单租户并发上限、token bucket、请求超时与取消、SSE 流式转发、重试、熔断、健康探针、优雅退出和 Prometheus 指标。测试覆盖分块 prefill、缓存续推、校验和损坏、TTL/LRU、租户隔离、vLLM 协议映射、Redis 分布式锁与准入控制。项目仓库同时提供冷/热路径基准脚本和并发负载脚本，可输出 TTFT、P95/P99、请求吞吐与 token 吞吐。

当前 0.3.0 公开版本提供基准测试工具，但不附带绑定特定模型、硬件和请求分布的结果表。因此，本文不把其他研究中的加速倍数写作本系统实测。Prompt Cache 在其特定原型与硬件上报告过

GPU 约 $8\times$ 、CPU 约 $60\times$ 的改善区间 [4], LMCACHE 也报告过特定工作负载下的吞吐提升 [5]; 这些数据只能说明技术潜力, 不能替代对 KV CACHE SERVICE 部署环境的独立测试。

4 系统效果与经济性分析

4.1 直接效果：降低重复计算与 TTFT

设一次公共前缀 prefill 的成本为 C_p , 缓存序列化与发布成本为 C_s , 一次加载成本为 C_l , 总共有 N 次使用。若不缓存, 总成本近似为 NC_p ; 若首次构建、后续复用, 总成本为

$$C_{\text{cache}} = C_p + C_s + (N - 1)C_l. \quad (5)$$

缓存产生正收益的条件为

$$(N - 1)(C_p - C_l) > C_s. \quad (6)$$

式(6)说明, 前缀越长、复用越多、存储越快, 缓存越有价值; 短前缀、低命中率或慢速远端存储则可能使加载成本高于重算。项目的主要效果可概括为:

- 对命中请求跳过公共前缀 prefill, 降低 TTFT 与 prefill GPU-seconds;
- 将模型内部状态显式化为可创建、查询、删除、统计和淘汰的服务对象;
- 通过亲和路由提高物理缓存命中概率, 并用分层存储扩大可保留的缓存集合;
- 通过准入、隔离、校验与观测能力, 使“缓存技巧”具备成为共享基础设施的条件。

4.2 效果不覆盖的部分

KV Cache 命中不会自动提高模型事实正确率, 也不会减少答案生成阶段的全部计算。它不能突破模型位置编码或上下文窗口, 不能把两个任意位置、任意模型生成的 KV 张量安全拼接, 也不能解决知识来源追踪。项目的 `chunk_size` 是分块执行 prefill 的工程参数, 并非 RAG 意义上的语义切块或可自由组合的知识块。若前缀的任一 token、chat template、模型权重、tokenizer、RoPE 参数、量化布局或张量并行拓扑发生变化, 原始 KV 都可能失效。

5 从 KV Cache 到企业知识资产

5.1 资产定义：从原始文档到模型绑定的派生制品

企业传统知识资产包括原始文档、结构化数据库、知识图谱、向量索引和规则库。持久化 KV Cache 增加了一种新的形态: 知识先被规范化为稳定提示词, 再由指定模型和 tokenizer 预解析为注意力状态。可将一个缓存资产形式化为

$$\mathcal{A} = \langle D, P, \tau, M, R, Q, \{K_l, V_l\}_{l=1}^L, G \rangle, \quad (7)$$

其中 D 为来源文档快照, P 为提示模板, τ 为精确 token 序列, M 为模型权重版本, R 为运行布局, Q 为权限与租户域, G 为校验、TTL、血缘等治理元数据。

与向量索引相比, KV Cache 更接近“已编译的知识制品”: 在线读取快, 但与编译器(模型)、指令集(tokenizer/布局)和源代码(文档)强绑定。它不应取代原始知识库, 而应被视为可再生的加速层。企业资产目录至少应记录来源哈希、模型指纹、模板版本、token 数、大小、创建时间、到期时间、所有者、敏感级别、授权域、预计复用率和最近命中时间。

5.2 价值：把重复理解转化为一次预解析

对于稳定制度、产品手册、代码基线、合同模板和 Agent 工具定义, 企业可以在内容发布时同步生成 KV Cache。此后, 多个用户对同一快照提问时, 模型无需反复“读完”相同材料。其价值不是把自然语言知识永久写入模型参数, 而是把一次模型相关的解析结果保存下来, 形成可审计、可过期、可重新构建的中间层资产。

这种资产尤其适合高复用、低更新、长前缀和权限边界清晰的知识。若知识频繁变更, 或不同用户只能看到文档的任意组合, 缓存数量会膨胀, 失效成本也会升高。此时应把缓存单位设计为版本化知识包, 而不是为每个临时查询保存完整状态。

6 长上下文问答：替代 RAG 的条件与边界

6.1 “绕过 token 限制”的严格解释

从 API 与推理成本角度看, 客户端在后续请求中只需提交 `cache_id` 与新增问题, 能够避免重复传输长文本, 并可能避免按重复输入 token 产生的在线处理成本; 从模型计算角度看, 缓存直接恢复内部状态, 能够跳过相同前缀的 `prefill`。因此, 它在工程上可以“绕过重复 token 的传输与计算”。

但从模型语义角度看, 缓存前缀仍占用上下文位置, 前缀 token 数与新增问题、输出预算之和仍不能超过模型窗口。KV CACHE SERVICE 源码会检测这一上限, 项目文档也明确指出分块 `prefill` 不能突破上下文窗口 [1]。故不能把 KV Cache 宣称为无限上下文技术。

6.2 无需 RAG 的适用条件

在以下条件同时满足时, 可考虑使用“完整知识前缀 + KV Cache”替代传统检索链路:

1. 知识快照能够连同系统提示和输出预算完整放入模型窗口;
2. 同一知识快照会被重复询问, 且前缀 token 序列可以保持稳定;
3. 业务更重视跨章节、跨文档的整体关系, 而不是只读取 Top- k 片段;
4. 知识更新频率较低, 可以接受内容变化后重建缓存;
5. 不要求检索系统天然提供的文档级来源、召回解释与超大语料扩展能力。

该方式的“全局知识”优势是: 模型在 `prefill` 时看到了完整前缀, 问答阶段不会因检索漏召回而完全丢失某一段材料, 也可利用跨章节关系。但是, 模型是否真正使用所有内容仍取决于长上下文能

力；LongBench 与 “Lost in the Middle” 等研究显示，窗口足够长不代表中间位置的信息会被同等可靠地利用 [7, 8]。

6.3 KV Cache、RAG 与混合方案比较

表 1: 三类企业知识问答方案的比较

维度	完整前缀 KV Cache	RAG	RAG + KV Cache 混合方案
知识规模	受模型窗口和 KV 存储限制	可扩展到大型外部语料	大语料先检索，热门结果或固定部分再缓存
在线成本	命中后省去公共前缀 prefill	有检索、重排和所选片段 prefill	同时减少检索后重复片段的 prefill
全局关系	完整上下文内较强，但受模型长文本能力影响	易遗漏未召回内容，跨片段关系依赖召回	在可控成本下兼顾召回与复用
知识更新	前缀变化通常需要重建	更新索引或文档较灵活	索引更新并使相关缓存失效
来源追踪	需额外设计引用映射	天然更易返回来源文档	由检索层提供引用
最佳场景	稳定、有限、反复询问的知识包	庞大、动态、问题相关性稀疏的语料	多数生产级企业知识平台

混合方案通常更稳妥：RAG 负责从大规模动态语料中选择上下文，KV Cache 负责复用固定系统前缀、热门知识包或重复检索结果。不过，独立预计算的多个文档块不能简单拼接而不考虑位置与跨块注意力；CacheBlend 等研究专门通过部分重计算处理这一问题 [9]，而 KV CACHE SERVICE 当前核心语义仍是精确公共前缀复用。

7 闲时算力向忙时能力的转换

7.1 时间迁移机制

传统在线推理要求在请求到达后立即完成全文 prefill。持久化 KV Cache 将其拆分为两个时间段：

1. **闲时**：根据知识发布事件、历史热度或预约任务，使用空闲 GPU/CPU 构建前缀状态，完成校验与持久化，并下沉到 CPU/NVMe；
2. **忙时**：根据 cache_id 和租户域查找状态，将热块加载到推理设备，仅计算问题后缀与答案解码。

这相当于把可预测的 prefill 计算从峰值时段迁到低谷时段，使峰值 GPU 更集中于动态后缀和 decode。对于日报、交易日制度、课程材料、代码版本发布等可预知知识快照，缓存构建可以直接纳入发布流水线。

7.2 调度策略

企业调度器不应无差别预热所有文档，而应以期望收益排序。令缓存 i 在计划周期内的预计命中次数为 h_i ，单次节约为 $C_{p,i} - C_{l,i}$ ，构建与存储成本为 $C_{s,i}$ ，则优先级可写为

$$U_i = h_i(C_{p,i} - C_{l,i}) - C_{s,i} - \lambda B_i, \quad (8)$$

其中 λ 表示容量影子价格。只有 $U_i > 0$ 的缓存才值得预热。高价值缓存放在 HBM/DRAM，次热缓存放在本地 NVMe，低频缓存可淘汰后按需重建。

“闲时算力存下来”并非字面保存算力，而是保存算力产生的模型中间状态。忙时仍需支付查找、校验、I/O、CPU-GPU 搬运、后缀 prefill 和 decode 成本；若缓存过大、网络存储过慢或热点预测错误，预计算可能只是把成本从计算转移到存储，并造成额外能耗。

8 缓存原生的场景工程

KV Cache 的收益依赖精确前缀复用，因此该技术会反向塑造应用和数据建设方式。理想的提示词布局为

$$\underbrace{\text{稳定系统指令} + \text{稳定知识快照} + \text{固定工具定义}}_{\text{可缓存公共前缀}} + \underbrace{\text{用户问题} + \text{实时变量}}_{\text{动态后缀}}. \quad (9)$$

企业可采用以下设计原则：

1. **前稳定、后动态**。将时间戳、随机 ID、用户问题和实时状态放到前缀之后，避免一个无关字符导致整体未命中。
2. **规范化序列化**。固定换行、空格、Unicode 归一化、JSON 字段顺序、工具列表顺序和 chat template；对严格边界使用 `input_ids`，避免分别分词造成 token 边界差异。
3. **版本化知识包**。以“制度版本、代码 commit、合同批次、产品手册版本”定义稳定快照，并将来源哈希映射到缓存 ID；内容更新采用新版本发布而非原地覆盖。
4. **按权限构建**。租户、部门和保密级别必须参与缓存命名与授权。不能为了提高命中率而让不同权限域共享可推断的缓存状态。
5. **为命中率设计流程**。在场景立项阶段估计前缀长度、访问频率、更新率、用户组合和存储带宽，优先建设高复用知识包，而非在上线后被动缓存所有请求。
6. **保留语义边界**。若多份材料之间高度相关，应以稳定顺序联合 prefill，保留跨文档注意力；若需要任意组合，应使用 RAG 或支持部分重计算的缓存融合技术，而非直接拼接独立 KV。

这意味着未来的“场景建设”不仅是写提示词，还包括知识编排、模板 ABI、模型版本、缓存目录、权限域和失效规则的共同设计。缓存命中率将成为与检索召回率同等重要的应用指标。

9 企业落地参考架构

一个可操作的企业流程可分为六层：

1. **知识源层**：文档库、代码库、数据库快照与工具定义；原始内容仍是事实来源；
2. **规范化层**：去除不稳定字段，统一模板、编码和排序，生成不可变知识快照；
3. **预解析层**：在闲时按目标模型分词和 prefill，生成 KV Cache、模型指纹与校验元数据；
4. **资产目录层**：维护来源血缘、权限、版本、TTL、热度、大小、命中率和重建策略；
5. **分层存储与路由层**：HBM/DRAM/NVMe/远端分层，配合缓存亲和、副本健康度、负载与熔断；
6. **在线服务层**：校验租户和版本，加载缓存，追加用户问题并生成答案，同时记录命中 token、TTFT 与引用信息。

KV CACHE SERVICE 已覆盖预解析、资产生命周期 API、部分目录元数据、分层部署参考与在线路由，但完整企业平台仍需补充统一身份系统、静态加密/KMS、文档血缘、审批审计、跨区域复制、缓存成本核算和知识更新事件总线。

10 评估方案

为验证项目在具体企业中的效果，应至少设置四组基线：完整长上下文冷请求、完整长上下文 KV 命中、RAG、RAG+KV 混合。实验变量包括前缀长度、模型规模、KV dtype、存储层级、并发度、生成长度、复用次数、知识更新频率与缓存命中率。建议报告：

- TTFT 的 P50/P95/P99、端到端延迟和每输出 token 延迟；
- prefill GPU-seconds、吞吐、峰值显存、CPU-GPU 与磁盘带宽；
- 缓存构建、加载、校验、命中、淘汰、过期和失败次数；
- 单次问答成本、单知识包盈亏平衡复用次数及单位存储节约的算力；
- 回答正确率、引用正确率、长文本不同位置的召回能力和版本时效性；
- 跨租户访问、损坏缓存、模型升级和副本故障等安全与韧性测试。

仓库自带 `scripts/benchmark.py` 可比较冷/热请求中位数，并拆分 `cache_load`、`prefill` 和总时延；`scripts/load_test.py` 可测量流式 TTFT、P95/P99 和吞吐。生产决策应基于真实模型、真实文档、真实存储和真实请求分布，而不是仅凭短文本或单次命中测试。

11 风险、限制与治理要求

1. **强兼容性耦合**。原始 KV 与模型权重、tokenizer、位置编码、dtype、量化、注意力布局和并行拓扑绑定；升级模型通常需要重建。
2. **存储放大**。KV 可能远大于原始文本或向量索引。缓存越长、模型越大、租户越多，容量压力越显著。
3. **精确前缀约束**。普通前缀缓存不能复用任意位置的相似语义；文本相同但 token 序列或位置不同也可能无法命中。
4. **知识时效性**。缓存会冻结某个知识快照。必须通过事件驱动失效、TTL 和版本路由防止旧制度继续服务。
5. **隐私与侧信道**。KV 张量不是匿名数据，应按原始提示词同等级别加密、授权和审计；共享推理

环境中的缓存命中时延也可能泄露前缀存在性。

6. **准确性边界**。完整上下文减少检索遗漏，但不会消除幻觉、长文本位置偏差和来源不可追踪问题。
7. **经济性不确定**。当 $T_{\text{load}} \geq T_{\text{prefill}}$ 、命中率低或缓存频繁失效时，持久化没有收益，应回退普通推理或 RAG。

12 结论

本文报告的 KV Cache Service 不只是一个磁盘缓存实现，而是将跨请求 KV 复用封装为具有稳定 API、内容身份、生命周期、租户隔离、容量治理、路由与可观测性的完整服务。Transformers 路径实现了原始 KV 的持久化与恢复，vLLM+LMCache 路径实现了面向多副本和分层存储的生产拓扑。系统能够减少重复长前缀 prefill 和 TTFT，并为闲时预热、忙时复用提供工程基础。

从更长期的产业视角看，KV Cache 可以成为企业知识体系中的“模型编译产物”：原始知识仍由文档库或数据库管理，KV Cache 作为与模型和版本绑定的派生资产，加速稳定知识的反复问答。它可在知识集合有限、上下文足够长、复用率高的场景中弱化甚至省去在线检索，并保留更完整的上下文；但它既不能突破模型 token 窗口，也不能全面替代 RAG。最可行的路线是缓存原生的场景工程与混合知识架构：用 RAG 管理大规模、动态和需引用的知识，用 KV Cache 管理稳定、高频和昂贵的公共上下文，并通过闲忙调度把可预测计算转化为可复用的服务能力。

参考文献

- [1] Nanamiqa. *KV Cache Service*: 面向本地与私有化大语言模型推理的持久化 KV Cache 服务, version 0.3.0, GitHub repository, 2026. <https://github.com/Nanamiqa/kvcache-service>. Accessed 2026-08-10.
- [2] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017. <https://arxiv.org/abs/1706.03762>.
- [3] W. Kwon, Z. Li, S. Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. <https://arxiv.org/abs/2309.06180>.
- [4] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, 2024. <https://arxiv.org/abs/2311.04934>.
- [5] Y. Liu, et al. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665, 2025. <https://arxiv.org/abs/2510.09665>.

- [6] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2005.11401>.
- [7] Y. Bai, X. Lv, J. Zhang, et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of ACL*, 2024. <https://arxiv.org/abs/2308.14508>.
- [8] N. F. Liu, K. Lin, J. Hewitt, et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. <https://arxiv.org/abs/2307.03172>.
- [9] J. Yao, H. Li, Y. Liu, et al. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444, 2025. <https://arxiv.org/abs/2405.16444>.